



University of Tehran

School of Electrical and Computer Engineering

GitHub Herd Behavior, Popularity Forecasting, and Repository Recommendation

Introduction to Data Science – Final Project Phase 3

Team Members

Arash Taherifard – Shayan Maleki – Ali Khosravi

Student IDs

810102474 – 810102515 – 810102606

Course instructors: Dr. Razavi and Dr. Yaghoobzadeh

Repository: [github-herd-data-pipeline](#)

Phase 3 branch: [github-herd-data-pipeline/tree/phase3](#)

Validated code commit:

8a864558bc8ea79543a88bd587948cda3a3df087

Abstract

This project develops an end-to-end data science system for studying collective attention and herd behavior in the GitHub open-source ecosystem. The dataset combines repository metadata, timestamped star interactions, user–repository relations, daily and weekly popularity time series, programming languages, topics, and engineered temporal and graph features. The final system addresses three connected tasks: forecasting repository star growth over the next four weeks, classifying repositories that are likely to experience a future growth surge, and generating top-10 herd-aware repository recommendations for users.

The implementation follows a leakage-aware scientific protocol. Candidate models were tuned only on training and validation data using repository-aware temporal splits, selected artifacts were frozen before the test sets were opened, and each final test was evaluated once. The selected forecasting system combines a rolling three-week baseline with an Extra Trees residual model. On the frozen test set it achieved an MAE of 127.05, RMSE of 277.97, RMSLE of 0.917, and sMAPE of 0.504. The selected calibrated logistic classifier achieved ROC-AUC 0.648, PR-AUC 0.398, recall 0.529, and F1 0.409 at its frozen validation-selected threshold. The final recommendation model combines item-based collaborative similarity and graph-based personalized PageRank with a popularity penalty. It achieved Recall@10 of 0.223, NDCG@10 of 0.118, catalog coverage of 0.924, long-tail share of 0.869, and intra-list diversity of 0.953 on the frozen test set.

The prediction pipeline stores repository forecasts, growth-surge probabilities, labels, and 1,511,420 user recommendation rows in a generated SQLite database. The system was additionally integrated with MLflow for frozen-model registration, Prefect for orchestration, Docker for portable execution, Kubernetes for a secured batch validation Job, and GitHub Actions for clean-checkout CI/CD. The project demonstrates that GitHub activity can support useful trend monitoring and personalized discovery, while also showing important limitations caused by rare viral growth, sparse implicit feedback, popularity bias, and the difference between public attention and software quality.

Executive Summary

What was built

A complete GitHub analytics and recommendation platform was implemented across three project phases. The system begins with GitHub REST API data collection, stores cleaned data in SQLite, performs reproducible preprocessing and feature engineering, trains and evaluates multiple candidate models, freezes selected artifacts, generates operational predictions, and validates delivery through modern MLOps tools.

Main outputs

- 82 repository-level operational prediction rows;
- 12 repositories classified as future growth surges;
- mean operational four-week forecast of 945.64 stars;
- 151,142 users served by the operational recommender;
- 145 repositories in the recommendation catalog;
- 10 recommendations per user and 1,511,420 expected recommendation rows;
- successful Prefect, MLflow, Docker, Kubernetes, and GitHub Actions validation.

How the results should be used

The models are appropriate for ranking, prioritization, ecosystem monitoring, and assisted discovery. They should not be treated as direct measures of software quality, guaranteed adoption, or fully automated investment decisions. Human review and additional information such as maintenance quality, security, licensing, documentation, and organizational fit remain necessary.

Contents

Abstract

Executive Summary

1	Project Definition, Motivation, and Requirements	1
1.1	Background	1
1.2	Problem statement	1
1.3	Research questions	1
1.4	Assignment requirements and implementation mapping	2
1.5	Project contributions	3
2	Dataset Collection and Construction	3
2.1	Data source and collection scope	3
2.2	Database entities and scale	3
2.3	Data dictionary and identifiers	4
2.4	Sampling and representativeness	4
2.5	Data quality risks	5
3	Database, Exploratory Analysis, and Preprocessing	5
3.1	SQLite implementation	5
3.2	Schema design	5
3.3	Exploratory data analysis	5
3.4	Missing-value treatment	6
3.5	Preprocessing decisions	7
3.6	Advanced feature engineering	7
3.6.1	Repository metadata features	7
3.6.2	Temporal and herd features	7
3.6.3	Recommendation features	8
3.6.4	Leakage prevention	8

4	Experimental Design and Scientific Protocol	8
4.1	Why ordinary random splitting is insufficient	8
4.2	Repository-level split design	9
4.3	Recommendation split design	9
4.4	Model search and selection	9
4.5	Metric selection	9
5	High-Growth Classification	10
5.1	Task definition	10
5.2	Candidate models	10
5.3	Temporal cross-validation and tuning	11
5.4	Selected model and calibration	11
5.5	Ablation and explainability	12
5.6	Frozen final-test results	12
5.7	Technical interpretation	14
5.8	Real-world interpretation	14
6	Four-Week Popularity Forecasting	14
6.1	Task definition	14
6.2	Initial model search	15
6.3	Selected baseline–residual model	15
6.4	Frozen final-test results	16
6.5	Technical interpretation	17
6.6	Business interpretation	18
7	Herd-Aware Repository Recommendation	18
7.1	Problem formulation	18
7.2	Candidate families	18
7.3	Selected scoring rule	18
7.4	Strict external validation and secondary branches	19
7.5	Frozen final-test results	20

7.6	Technical interpretation	20
7.7	Real-world interpretation	20
8	Operational Prediction Pipeline and Database Integration	21
8.1	Separation of training and prediction	21
8.2	Operational inference sequence	21
8.3	Operational output scale	21
8.4	Database write policy	22
8.5	Read-only delivery validation	22
9	MLOps, Automation, and Deployment	22
9.1	Phase 2 pipeline and GitHub Actions	23
9.2	Prefect orchestration	23
9.3	MLflow model management	23
9.4	Docker packaging	24
9.5	Kubernetes validation	24
9.6	Phase 3 GitHub Actions	24
9.7	Benefits and shortcomings of the automation choices	25
9.7.1	Benefits	25
9.7.2	Shortcomings	25
10	Integrated Technical and Business Analysis	26
10.1	What the three tasks provide together	26
10.2	Stakeholder analysis	26
10.3	Business value	26
10.4	Cost of errors	27
10.5	Popularity feedback loops	27
10.6	Responsible interpretation	27
11	Limitations, Risks, and Future Work	27
11.1	Data limitations	27

11.2	Modeling limitations	28
11.3	Deployment limitations	28
11.4	Ethical and ecosystem risks	28
11.5	Future work	28
12	Conclusion	29
A	Reproducibility Commands	30
A.1	Main Phase 2 pipeline	30
A.2	Phase 3 model-development workflow	30
A.3	Portable read-only Phase 3 validation	30
A.4	Operational validation without writes	30
A.5	Write-enabled operational prediction	31
A.6	Docker	31
A.7	Kubernetes	31
A.8	Prefect	32
A.9	MLflow	32
B	Frozen Artifacts and Git References	33
C	Repository Structure	34
	References	35

List of Figures

1	Exploratory analysis of repository popularity and growth dynamics.	6
2	Permutation importance of the selected classifier on validation data.	12
3	Frozen classification discrimination curves.	13
4	Frozen classification threshold and calibration diagnostics.	14
5	Frozen forecasting predictions and residuals.	17
6	Forecasting benchmark and repository-level heterogeneity.	17
7	Recommendation relevance and coverage trade-off.	19
8	Separation of the training and operational prediction pipelines.	21

List of Tables

1	Assignment requirements and project implementation.	2
2	Main SQLite tables and row counts.	4
3	Primary metrics by task.	10
4	Classification model-family comparison before probability calibration.	11
5	Frozen final classification test metrics.	13
6	Selected results from the initial forecasting validation search.	15
7	Frozen final forecasting test metrics.	16
8	Strict external validation of primary and secondary recommendation branches.	19
9	Frozen final recommendation test metrics.	20
10	Validated operational prediction outputs.	22
11	Validated automation and deployment components.	25
12	Potential stakeholders and decisions supported by the system.	26
13	Frozen artifact SHA-256 hashes.	33

1 Project Definition, Motivation, and Requirements

1.1 Background

Open-source software has become a central part of modern software development. GitHub provides public traces of this ecosystem through repositories, stars, forks, topics, languages, issues, and user interactions. Among these signals, stars are frequently used as a simple measure of attention. Their interpretation is not straightforward: a star may represent interest, bookmarking, planned use, actual adoption, or social imitation. A repository that becomes visible can receive more stars because of its visibility itself, creating a self-reinforcing process similar to herd behavior.

This project studies that process from three complementary perspectives. Forecasting asks how much attention a repository may receive in the near future. Classification asks whether the repository is likely to enter a high-growth regime. Recommendation asks which unseen repositories should be presented to each user and whether the system can avoid concentrating exposure only on the already-popular head of the catalog.

1.2 Problem statement

The central problem is:

Can repository metadata, temporal growth patterns, user interactions, and graph structure be combined to forecast future popularity, detect growth surges, and generate useful recommendations without simply reproducing global popularity?

This problem is technically challenging because repository growth is highly skewed, viral events are rare, the user–repository interaction matrix is sparse, observations are time dependent, and popular items are easier to recommend than relevant long-tail items.

1.3 Research questions

1. How accurately can future four-week repository star growth be forecast from historical information available at a cutoff date?
2. Can a model distinguish repositories likely to experience future growth surges?
3. Which model families generalize best under repository-aware temporal validation?
4. Can collaborative and graph signals produce useful top-10 repository recommendations?
5. How does a popularity penalty affect recommendation coverage, novelty, diversity, and long-tail exposure?
6. Can the complete workflow be automated, tracked, packaged, and validated reproducibly?

1.4 Assignment requirements and implementation mapping

The Phase 3 brief requires multiple-model testing, train/validation/test separation, tuning and regularization, task-appropriate metrics, separate training and prediction pipelines, persistence of predictions to a database, and clear technical and real-world analysis [1].

Table 1. Assignment requirements and project implementation.

Requirement	Implementation	Evidence or location
Multiple candidate models	Classification, forecasting, and recommendation model families were compared under frozen selection rules.	outputs/phase3/*/
Train, validation, and test sets	Repository-aware temporal splits for repository tasks and time-aware held-out interactions for recommendation.	data/modeling/
Hyperparameter tuning	Grid searches, temporal cross-validation, regularization, threshold selection, and internal recommendation tuning.	Training scripts and search CSV files
Appropriate metrics	Classification discrimination/calibration metrics, forecasting error metrics, and top- K ranking, coverage, novelty, and diversity metrics.	Final-test artifacts
Separate train and predict pipelines	Development scripts train and freeze artifacts; operational scripts load those artifacts without retraining.	scripts/
Save predictions to database	Repository predictions and ranked recommendations are written to an isolated generated SQLite database.	database/generated/
End-to-end automation	Phase 2 pipeline, portable Phase 3 runner, Prefect flow, Docker, Kubernetes, and GitHub Actions.	pipeline.py, run_phase3.sh
Prefect bonus	Read-only flow, deployment, process worker, and completed deployment-triggered run.	flows/, prefect.yaml
MLflow bonus	Frozen models, metrics, signatures, versions, aliases, and operational evidence were registered.	outputs/phase3/mlflow/

Requirement	Implementation	Evidence or location
Business interpretation	Stakeholder use cases, error costs, value propositions, risks, and responsible-use limits are analyzed.	Sections 10 and 11

1.5 Project contributions

The project contributes:

- a reproducible GitHub repository and interaction dataset;
- a normalized SQLite database with relational, temporal, and modeling tables;
- advanced temporal, herd, content, and recommendation features;
- three task-specific modeling systems with strict test-set protection;
- a high-coverage, long-tail-aware recommendation strategy;
- operational database outputs for forecasts, classifications, and recommendations;
- model registration and orchestration using MLflow and Prefect;
- hardened container and Kubernetes validation;
- clean-checkout GitHub Actions validation and hash-based evidence.

2 Dataset Collection and Construction

2.1 Data source and collection scope

The dataset was collected through the GitHub REST API [2].

The collection design intentionally supports both temporal prediction and recommendation. Repository metadata provides relatively static descriptors; timestamped stars provide behavioral events; daily and weekly aggregates provide growth trajectories; and user–repository interactions provide the sparse implicit-feedback matrix.

2.2 Database entities and scale

The final Phase 2 database contains six principal tables. Their sizes are shown in Table 2.

Table 2. Main SQLite tables and row counts.

Table	Purpose	Rows
repositories	Repository and owner meta-data	763
stars	Raw timestamped stargazer events	216,566
user_repo_interactions	Positive implicit-feedback interactions	216,566
daily_timeseries	Daily repository star counts	11,128
weekly_timeseries	Weekly repository star counts	2,250
herd_modeling	Repository-level herd modeling table	145

The recommendation catalog contains 145 repositories and 151,142 distinct users. The interaction split contains approximately 189,964 training events, 13,301 validation events, and 13,301 test events, preserving the complete total of 216,566 interactions.

2.3 Data dictionary and identifiers

The central identifier is `repo_id`. It links repository metadata, star events, time-series records, and herd-modeling features. The recommendation data additionally uses `user_id`. Human-readable fields such as `repo_full_name` and `username` are retained for interpretation, while numeric identifiers are used for joins and matrix construction.

Important timestamps include repository creation and update dates, individual `starred_at` events, weekly cutoff dates, and target-end dates. Timezone-aware parsing is used so that temporal order remains consistent.

2.4 Sampling and representativeness

The data is large enough for a substantial implicit-feedback recommendation problem, but the repository-level modeling sample is much smaller. Only repositories with sufficient clean historical coverage can enter the leakage-safe forecasting and classification datasets. Consequently, the repository-level final tests represent 22 repositories rather than the entire database. This distinction is important: user-level recommendation estimates are based on many users, whereas repository forecasting uncertainty is dominated by a much smaller number of independent repository trajectories.

2.5 Data quality risks

The main data-quality risks are:

- incomplete repository metadata, especially descriptions, homepage URLs, licenses, and languages;
- rate limiting, pagination constraints, and interrupted API transfers;
- deleted users or repositories;
- stars generated by bots, campaigns, or coordinated promotion;
- stars as an attention signal rather than direct evidence of installation or long-term use;
- selection bias toward repositories discoverable through the chosen topics.

3 Database, Exploratory Analysis, and Preprocessing

3.1 SQLite implementation

SQLite was selected because it is serverless, portable, easy to inspect, and suitable for a course project. The tracked database is located at:

```
database/github_herd.db
```

The connection module uses `pathlib.Path` and relative paths rather than machine-specific absolute directories. Import and SQL validation were automated through modular scripts. The full Phase 2 pipeline imports source tables, executes validation queries, loads tables into pandas, performs EDA, preprocesses values, and generates model-ready features.

3.2 Schema design

The schema separates repository entities, event-level stars, user interactions, aggregated time series, and herd-modeling records. This avoids storing repeated metadata in every event row and makes task-specific queries efficient. The relational structure also supports integrity checks such as duplicate-key detection, null inspection, date-range validation, and row-count reconciliation.

3.3 Exploratory data analysis

The exploratory analysis generated dataset summaries, missing-value reports, language distributions, star and fork histograms, class-balance summaries, and selected weekly growth plots. The most important observations were:

- repository stars and forks have heavy-tailed distributions;

- a small number of repositories account for a large share of attention;
- weekly star growth is bursty rather than smooth;
- repository metadata is incomplete but event and identifier fields are mostly complete;
- the original repository-level herd label was nearly balanced, with 72 negatives and 73 positives;
- repositories that grew later often had much lower early popularity, illustrating why current size alone is not sufficient for detecting emerging growth.

The earlier Phase 2 aggregation showed average early four-week stars of 1,460.83 for the non-later-growth class versus 187.88 for the later-growth class, while their average later stars were 39.17 and 1,299.33 respectively. This descriptive contrast motivated the use of growth dynamics rather than static popularity alone.

Figure 1 illustrates two central exploratory findings. Repository popularity is strongly heavy-tailed, while the comparison of early and later stars shows that repositories with low initial attention can later become high-growth projects.

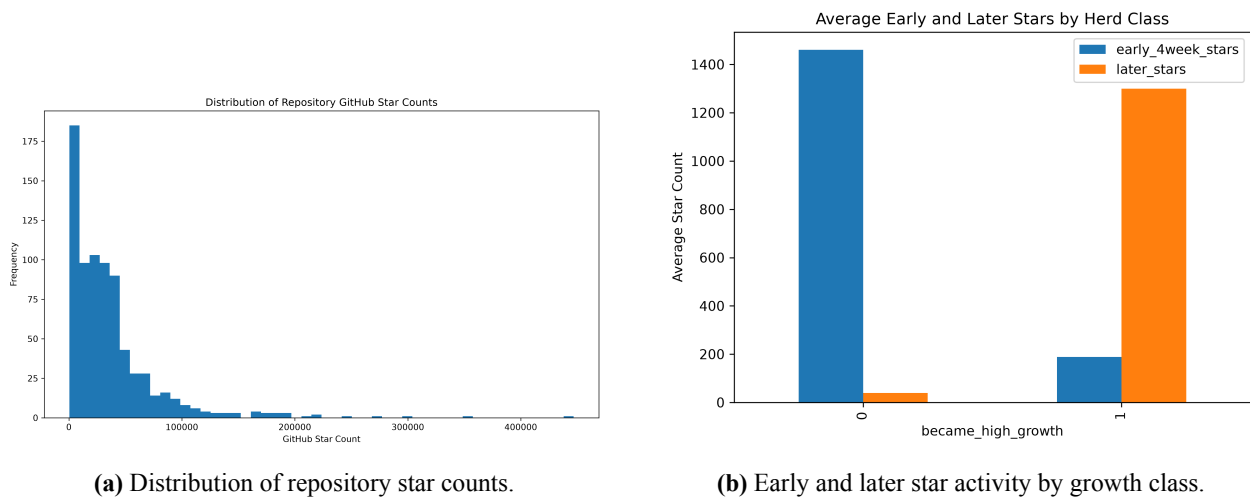


Figure 1. Exploratory analysis of repository popularity and growth dynamics.

3.4 Missing-value treatment

Missing repository descriptions, languages, licenses, and homepage URLs were expected because these fields are optional on GitHub. Textual missing values were filled with explicit placeholders where a model required a category. Numeric counts were converted to valid numeric types and checked for impossible negative values. Date fields were parsed with explicit error handling. Event-level tables were checked for missing identifiers and timestamps because such rows could not safely participate in temporal splits.

3.5 Preprocessing decisions

The main preprocessing operations were:

1. remove duplicate repository and interaction keys;
2. standardize timestamps and sort records chronologically;
3. coerce numeric columns and reject non-finite values;
4. fill or encode optional categorical metadata;
5. create repository age and basic count-ratio features;
6. aggregate stars into daily and weekly windows;
7. construct sparse binary user–repository matrices;
8. keep the prediction target isolated from feature-generation code;
9. save processed outputs in task-specific directories.

3.6 Advanced feature engineering

Feature engineering was designed around four groups.

3.6.1 Repository metadata features

These include stars, forks, watchers, repository age, owner type, language, topic count, and other static descriptors. Their role is to capture structural differences among projects, but they are treated cautiously because current popularity can dominate weaker signals.

3.6.2 Temporal and herd features

Weekly features include:

- current and previous weekly stars;
- second lag of weekly stars;
- rolling three-week mean;
- rolling four-week total;
- week-over-week growth rate;
- growth acceleration;
- cumulative popularity;

- clean history length;
- momentum and surge-related indicators.

These features separate level, trend, acceleration, and volatility. Only values available at a prediction cutoff are used.

3.6.3 Recommendation features

Recommendation development used:

- binary user–repository history;
- item co-occurrence cosine similarity;
- graph-based personalized PageRank scores;
- truncated-SVD latent factors;
- TF–IDF content from repository text and topics;
- static language and topic similarity;
- popularity counts and novelty scores;
- long-tail membership;
- cutoff-aligned forecast signals in a secondary branch.

3.6.4 Leakage prevention

For a row with cutoff time t , every feature must be computable using data available no later than t . Future star totals and future growth labels are only targets. Recommendation held-out positives are removed from each user’s fitting history, and the scoring code explicitly masks already-seen repositories. This constraint is more important than maximizing an optimistic offline score.

4 Experimental Design and Scientific Protocol

4.1 Why ordinary random splitting is insufficient

Random row splitting would mix future and past observations from the same repositories and allow temporally adjacent records to appear in both training and validation. This would overestimate performance. Therefore, repository-level tasks use time-respecting folds and external validation periods, while recommendation uses chronological held-out interactions.

4.2 Repository-level split design

Forecasting development used 700 training rows from 22 repositories, 66 external-validation rows from the same 22 repositories, and a sealed 66-row final test set. Each forecast predicts a 28-day target period. Classification follows the same repository-aware temporal structure and uses the binary future-growth-surge target.

4.3 Recommendation split design

For recommendation, interactions are ordered by time per user. Training history is used to construct item and graph relationships. Validation and test each contain one held-out positive interaction for eligible users. A candidate is valid only if it belongs to the 145-repository catalog and has not appeared in the user's fitting history.

4.4 Model search and selection

The general protocol was:

1. define task and split contracts before model search;
2. search candidate families using only training folds;
3. compare family winners on an external validation set;
4. choose thresholds and hyperparameters on validation only;
5. audit target leakage, temporal assumptions, and reproducibility;
6. serialize selected artifacts and record SHA-256 hashes;
7. create a Git freeze tag before loading the final test set;
8. evaluate the test once and prohibit post-test retuning.

Scientific freeze rule

The final-test results are descriptive estimates of the already-selected systems. A baseline that happens to perform better on the test cannot replace the frozen model after the test is opened. This avoids test-set adaptation and preserves the validity of the evaluation.

4.5 Metric selection

Classification is evaluated using discrimination, threshold, and calibration metrics. Forecasting uses absolute, squared, log-scale, percentage, and bias metrics. Recommendation uses ranking accuracy as well as catalog-level behavior because a recommender can achieve relevance by repeatedly recommending only popular repositories.

Table 3. Primary metrics by task.

Task	Metrics
Classification	ROC-AUC, PR-AUC, accuracy, balanced accuracy, precision, recall, F1, MCC, Cohen’s kappa, Brier score, log loss, calibration error, confusion matrix
Forecasting	MAE, RMSE, RMSLE, sMAPE, median absolute error, maximum absolute error, forecast bias
Recommendation	Precision@10, Recall@10, Hit Rate@10, NDCG@10, MAP@10, MRR, mean positive rank, catalog coverage, novelty, long-tail share, intra-list diversity

5 High-Growth Classification

5.1 Task definition

The classification target is `future_growth_surge`. The model returns a probability that a repository will experience a high-growth event in the target period. This probability supports ranking and prioritization; the final binary label is obtained by comparing it with a validation-selected threshold.

5.2 Candidate models

The corrected classification search evaluated a dummy baseline, logistic regression, K-nearest neighbors, decision trees, random forests, gradient boosting, histogram gradient boosting, and XGBoost. Most estimators and evaluation tools were implemented with `scikit-learn` [3], while the XGBoost candidate used the dedicated XGBoost implementation [4]. Numeric features were standardized where required. Class weighting and XGBoost scale weighting were included in the tuning grids. The primary refit metric was temporal-CV ROC-AUC, while external validation ROC-AUC and PR-AUC determined the family winner.

Table 4 summarizes the best configuration from each candidate family. Although gradient boosting achieved the highest mean temporal-CV ROC-AUC, logistic regression achieved the strongest external-validation ROC-AUC and PR-AUC. The predefined selection rule therefore chose logistic regression.

Table 4. Classification model-family comparison before probability calibration.

Model	CV ROC-AUC	CV PR-AUC	Validation ROC-AUC	Validation PR-AUC
Logistic regression	0.7358	0.5330	0.7363	0.4737
Histogram gradient boosting	0.7101	0.5187	0.6563	0.4247
XGBoost	0.7370	0.5408	0.6413	0.4373
Decision tree	0.7280	0.4930	0.6250	0.3212
Gradient boosting	0.7494	0.5597	0.6213	0.4367
Random forest	0.7286	0.5162	0.6113	0.4341
K-nearest neighbors	0.6357	0.4449	0.5825	0.3941
Dummy prior baseline	0.5000	0.2879	0.5000	0.2424

5.3 Temporal cross-validation and tuning

Repository-aware temporal folds ensured that each validation block occurred after its corresponding training block. Grid search recorded ROC-AUC, PR-AUC, accuracy, balanced accuracy, precision, recall, F1, MCC, Cohen’s kappa, log loss, and Brier score. The selected threshold maximized validation MCC, with balanced accuracy and F1 used as tie breakers.

5.4 Selected model and calibration

The selected base estimator is a scikit-learn pipeline containing standardization followed by logistic regression. The estimator uses the `saga` solver, a maximum of 10,000 iterations, `C = 10.0`, `l1_ratio = 0.5`, no class weighting, and random state 42. These settings were selected through a grid search over three repository-aware temporal folds.

After model-family selection, probability calibration was applied using training-only temporal-CV sigmoid calibration. The calibrated version was selected using validation log loss, Brier score, expected calibration error, and a constraint on discrimination loss. Isotonic calibration was evaluated diagnostically but was not deployment eligible because the temporal calibration folds were small.

The final threshold was selected on validation data by maximizing MCC, followed by balanced accuracy and F1 as tie breakers. The frozen threshold is:

$$\tau = 0.1748254577.$$

For feature vector $\mathbf{x}$, logistic regression estimates:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The operational label is $\hat{y} = 1$ when the calibrated probability is at least τ .

5.5 Ablation and explainability

A target-related feature was removed in a dedicated ablation to test whether performance depended excessively on a single engineered signal. Permutation importance was computed on validation data using the decrease in ROC-AUC after repeated feature shuffling. These analyses were completed before the test set was opened.

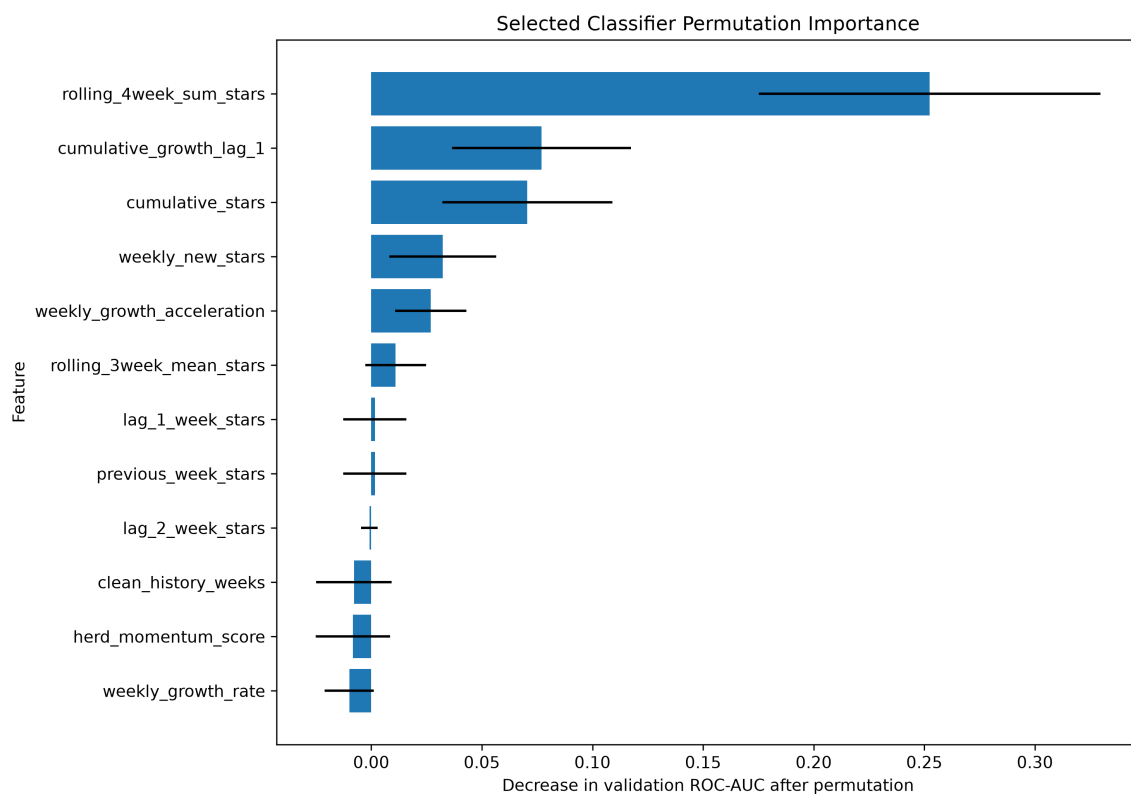


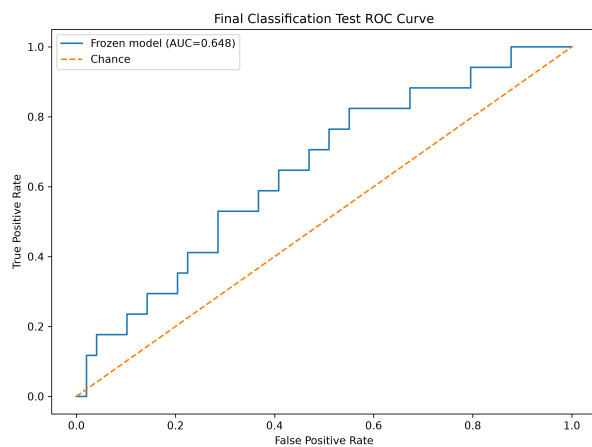
Figure 2. Permutation importance of the selected classifier on validation data.

5.6 Frozen final-test results

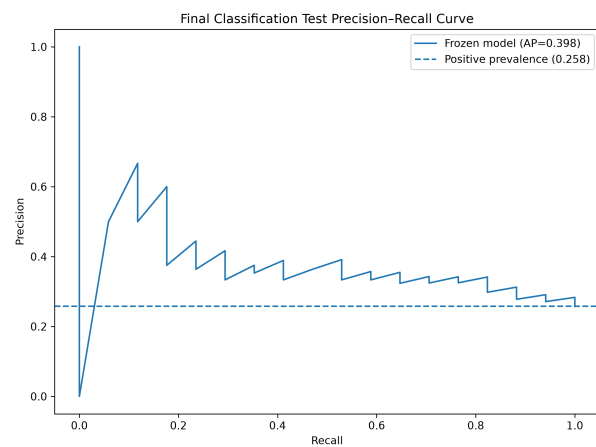
The final test contains 66 rows from 22 repositories. Table 5 reports the one-time frozen-threshold results.

Table 5. Frozen final classification test metrics.

Metric	Value
Accuracy	0.6061
Balanced accuracy	0.5810
Precision	0.3333
Recall	0.5294
F1 score	0.4091
ROC-AUC	0.6483
PR-AUC	0.3983
MCC	0.1441
Brier score	0.1890
Log loss	0.5705
True negatives	31
False positives	18
False negatives	8
True positives	9



(a) ROC curve.



(b) Precision-recall curve.

Figure 3. Frozen classification discrimination curves.

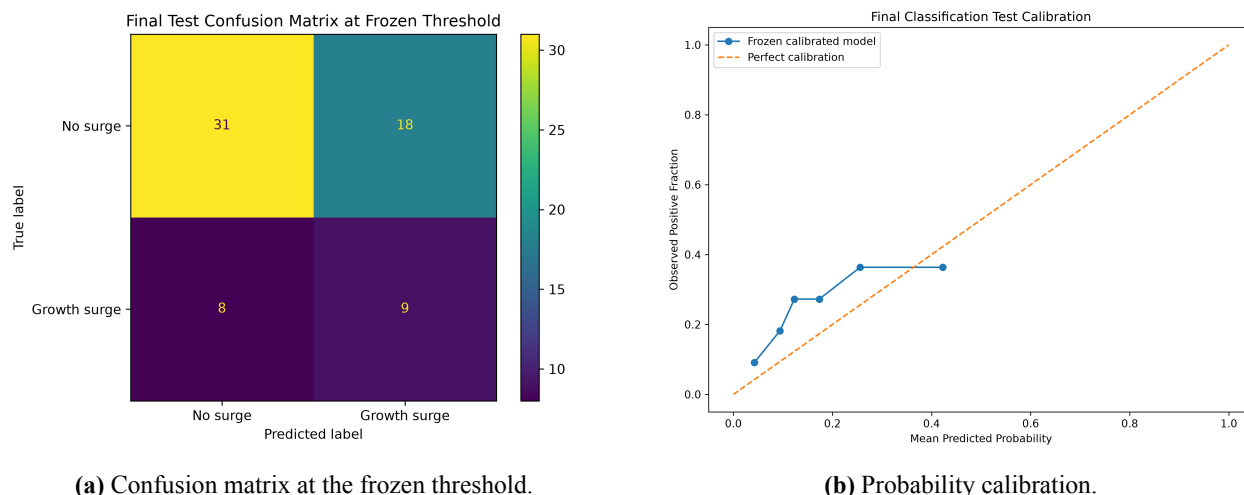


Figure 4. Frozen classification threshold and calibration diagnostics.

5.7 Technical interpretation

The ROC-AUC of 0.648 indicates moderate ranking information: a randomly selected future-surge row receives a higher score than a randomly selected non-surge row more often than chance, but the separation is not strong. PR-AUC is more informative under positive-class rarity and shows that the model improves on an uninformed ranking but still produces many false alarms.

At the low frozen threshold, recall is 0.529 and precision is 0.333. This trade-off was intentional: the classifier acts more like an early-warning filter than a strict confirmation system. Eighteen false positives indicate that not every flagged repository actually surges, while eight false negatives show that some emerging repositories remain difficult to detect.

5.8 Real-world interpretation

For trend-monitoring teams, missing an emerging repository may be more costly than reviewing an extra false alarm. Therefore, moderate recall at a lower threshold can be useful when the output feeds a human analyst dashboard. By contrast, an automated purchasing or investment rule would require substantially better calibration, external validation on a broader repository universe, and explicit cost-sensitive threshold design.

6 Four-Week Popularity Forecasting

6.1 Task definition

The regression target is `future_4week_stars`, defined as the number of stars received during the 28 days following a cutoff week. The prediction must be non-negative and must use only information available at the cutoff.

6.2 Initial model search

Twelve model families and 395 hyperparameter/target-transform configurations were evaluated using three temporal CV folds. The families were dummy mean, dummy median, ridge, elastic net, Poisson regression, KNN, decision tree, random forest, Extra Trees, gradient boosting, histogram gradient boosting, and XGBoost.

The first external-validation comparison exposed an important result: a simple rolling baseline outperformed every direct learned model on validation RMSLE. Selected rows are shown in Table 6.

Table 6. Selected results from the initial forecasting validation search.

Approach	RMSLE	MAE	RMSE	sMAPE
Rolling 3-week mean $\times 4$	0.5162	50.67	112.86	0.3796
Previous 4-week total	0.5453	52.71	118.81	0.3841
Extra Trees direct model	0.5935	57.29	126.41	0.4077
Random forest direct model	0.5963	57.44	127.37	0.4006
XGBoost direct model	0.6002	58.59	127.61	0.4162
Gradient boosting direct model	0.6020	59.00	129.63	0.4016

This result prevented the project from selecting a complex model merely because it was complex. Instead, the baseline became the center of a second-stage residual-learning audit.

6.3 Selected baseline–residual model

The final approach is:

$$\hat{y}_{r,t} = 4 \bar{s}_{r,t}^{(3)} + \hat{e}_{r,t},$$

where $\bar{s}_{r,t}^{(3)}$ is the rolling three-week mean of weekly stars and $\hat{e}_{r,t}$ is an Extra Trees prediction of the baseline residual. Predictions are clipped at zero.

The residual estimator uses 400 trees with maximum depth 4, `max_features=sqrt`, and minimum leaf size 10. The frozen recent-history feature set is:

- `weekly_new_stars`;
- `previous_week_stars`;
- `weekly_growth_rate`;
- `lag_2_week_stars`;
- `rolling_3week_mean_stars`;
- `rolling_4week_sum_stars`;
- `weekly_growth_acceleration`.

This architecture preserves the strong deterministic baseline while allowing a nonlinear model to correct systematic under- or over-prediction.

6.4 Frozen final-test results

The final test contains 66 observations from 22 repositories. Table 7 presents the frozen model metrics.

Table 7. Frozen final forecasting test metrics.

Metric	Value
MAE	127.0481
RMSE	277.9668
RMSLE	0.9174
sMAPE	0.5038
Median absolute error	27.2615
Maximum absolute error	1,208.8551
Forecast bias	-88.8777

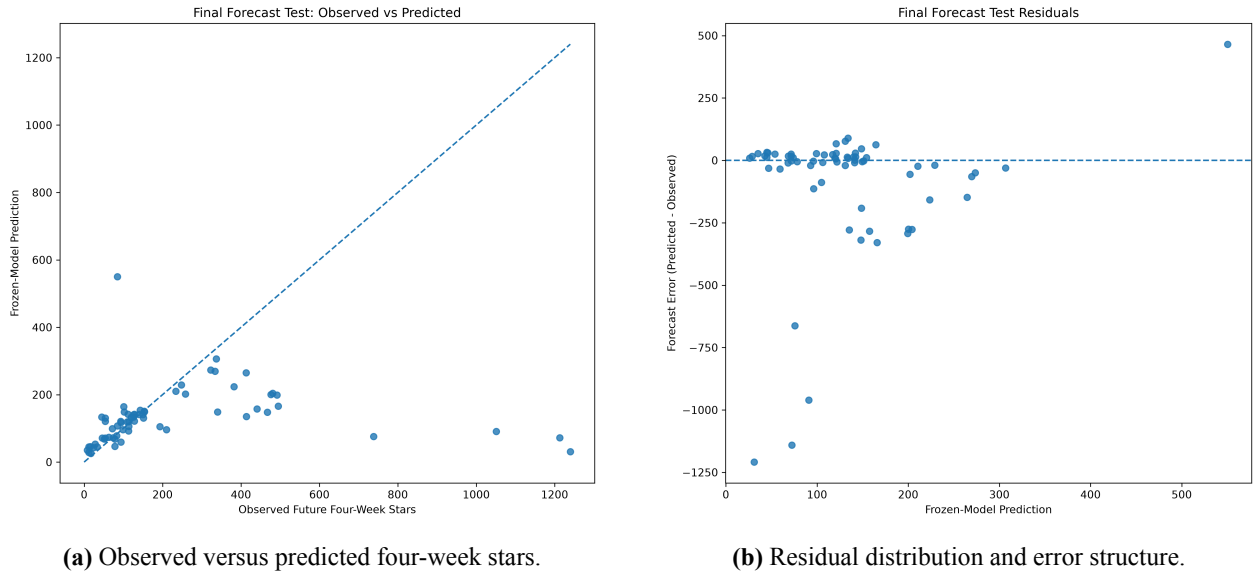


Figure 5. Frozen forecasting predictions and residuals.

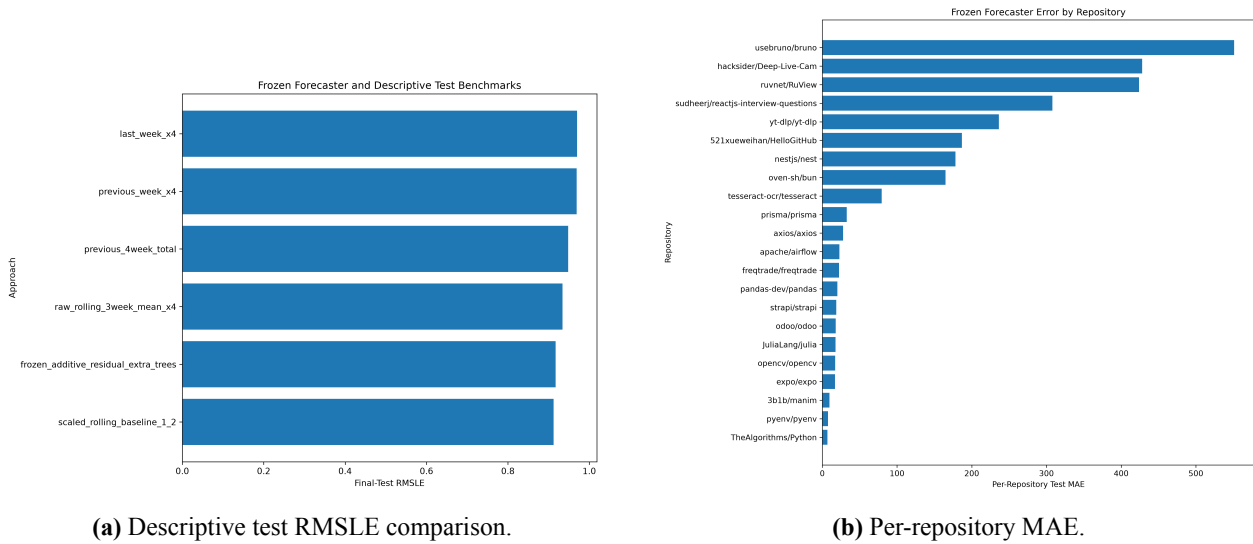


Figure 6. Forecasting benchmark and repository-level heterogeneity.

6.5 Technical interpretation

The median absolute error of 27.26 is much smaller than the MAE of 127.05 and the maximum error above 1,200. This indicates a heavy-tailed error distribution in which most forecasts are substantially better than the aggregate MAE suggests, but a few viral or abrupt growth cases dominate RMSE and bias.

Forecast bias is negative, so the model underpredicts on average. This is expected when sudden bursts cannot be inferred from the repository's recent history. The result also explains the deterioration from validation to test RMSLE: the test period likely contains a different mixture of growth regimes or more severe tail events.

6.6 Business interpretation

Forecasts can support capacity planning, trend dashboards, and technology scouting. A maintainer of a repository predicted to grow may prepare documentation, triage processes, community support, or infrastructure. An engineering organization may use forecasts to prioritize technologies for manual review. However, the large tail errors mean that forecast intervals and alerts should be preferred over single-point commitments.

7 Herd-Aware Repository Recommendation

7.1 Problem formulation

For user u , the recommender ranks candidate repositories r that are absent from the user's fitting history. Each validation or test user has one held-out positive repository. The primary relevance question is whether that repository appears near the top of the ranked list.

7.2 Candidate families

Development evaluated:

- popularity baselines;
- item–item collaborative similarity;
- graph-based personalized PageRank;
- truncated-SVD latent-factor scoring;
- collaborative–graph hybrids;
- popularity-penalized herd-mitigation rerankers;
- TF–IDF content models;
- static content hybrids;
- cutoff-aligned forecast reranking.

Internal tuning selected one configuration per family. Family winners were then compared on a strict external validation cohort of 13,295 users.

7.3 Selected scoring rule

The selected primary model is `herd_mitigation_penalty_0.05`. Its relevance score combines item-based collaborative and graph signals:

$$S_{\text{base}}(u, r) = 0.75 S_{\text{item}}(u, r) + 0.25 S_{\text{graph}}(u, r).$$

Candidate scores are rank normalized and adjusted with a popularity penalty:

$$S_{\text{final}}(u, r) = S_{\text{base}}^{\text{rank}}(u, r) - 0.05 P(r),$$

where $P(r)$ is a normalized repository-popularity quantity. Seen items receive $-\infty$ and cannot be recommended. Stable sorting by repository identifier guarantees deterministic ties.

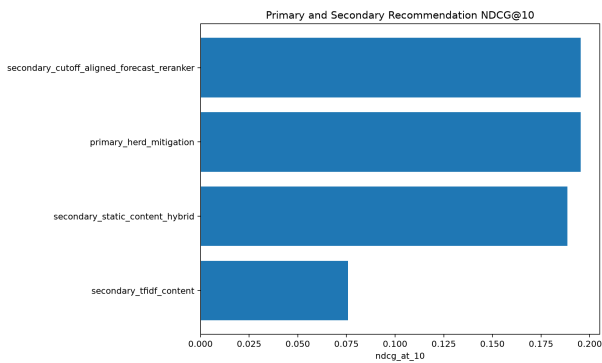
7.4 Strict external validation and secondary branches

Table 8 compares the primary model with secondary content and forecast branches. A forecast weight of zero was selected by internal tuning, so the forecast reranker exactly matches the primary model on strict validation. Static content slightly reduced ranking accuracy while preserving diversity. Pure TF-IDF content obtained excellent coverage but substantially weaker relevance.

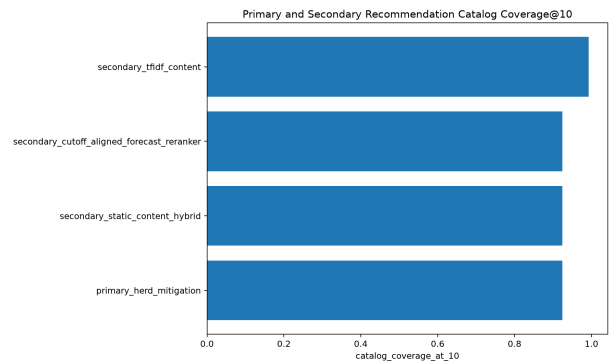
Table 8. Strict external validation of primary and secondary recommendation branches.

Model	NDCG@10	MRR	Recall@10	Coverage@10	Novelty@10	Long-tail share
Primary herd mitigation	0.1954	0.1665	0.3657	0.9241	7.2130	0.9241
Forecast reranker, weight 0	0.1954	0.1665	0.3657	0.9241	7.2130	0.9241
Static content hybrid	0.1885	0.1584	0.3639	0.9241	7.2121	0.9267
TF-IDF content	0.0758	0.0734	0.1601	0.9931	7.1937	0.8563

Paired bootstrap analysis with 2,000 replications showed that the static hybrid had lower NDCG@10 and MRR than the primary model, while its Recall@10 difference crossed zero. The TF-IDF model was decisively worse on all three ranking metrics. These results justify retaining the simpler primary system.



(a) NDCG@10 across strict-validation branches.



(b) Catalog coverage across strict-validation branches.

Figure 7. Recommendation relevance and coverage trade-off.

7.5 Frozen final-test results

The frozen test evaluates 13,301 users against a 145-repository catalog. Table 9 summarizes the results.

Table 9. Frozen final recommendation test metrics.

Metric	Value
Precision@10	0.02227
Recall@10	0.22269
Hit Rate@10	0.22269
NDCG@10	0.11779
MAP@10	0.08638
MRR	0.10923
Catalog Coverage@10	0.92414
Long-tail share@10	0.86851
Intra-list diversity@10	0.95312
Novelty@10	7.18350
Mean positive rank	43.7935
Median positive rank	33

7.6 Technical interpretation

With exactly one held-out positive per user, Precision@10 equals Recall@10 divided by ten. Recall@10 of 0.223 means that the held-out repository appears in the first ten results for approximately 22.3% of users. NDCG and MRR reward earlier positions and show that many recovered positives are not at rank one.

The catalog metrics are an important strength. Coverage above 92% means the recommender uses most of the available catalog. A long-tail share near 87% and intra-list diversity above 95% show that top-10 lists are not dominated by near-duplicate popular items. The validation-to-test reduction in relevance indicates cohort or temporal drift and reinforces the need for continued monitoring.

7.7 Real-world interpretation

A platform could use this system to broaden repository discovery and give relevant long-tail projects an opportunity to be seen. The popularity penalty directly addresses a business and ecosystem risk: if recommendation exposure is concentrated on the head, popular projects become even more popular and the platform loses discovery value. The chosen model trades some raw relevance for breadth and diversity, which may improve user exploration and ecosystem health.

8 Operational Prediction Pipeline and Database Integration

8.1 Separation of training and prediction

The assignment requires two pipelines. The training pipeline creates and freezes artifacts. The prediction pipeline loads those artifacts and applies identical feature contracts to current data. It never fits on prediction data.

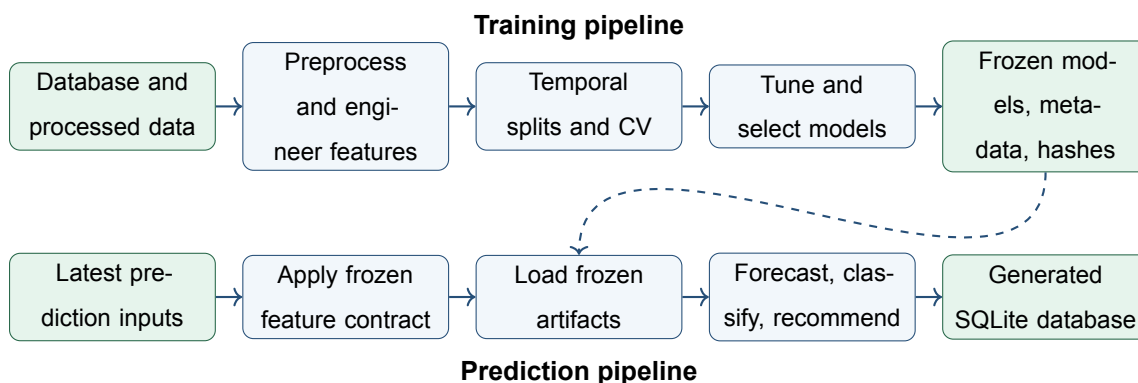


Figure 8. Separation of the training and operational prediction pipelines.

8.2 Operational inference sequence

The operational script performs:

1. verify required artifacts, metadata, and input columns;
2. select the most recent complete weekly feature row for each eligible repository;
3. generate four-week forecasts using the frozen baseline–residual model;
4. generate calibrated growth-surge probabilities and labels;
5. fit the recommendation history using train plus validation interactions;
6. score unseen candidates in batches;
7. write top-10 recommendations and repository predictions to a separate SQLite database;
8. validate row counts, keys, ranks, seen-item exclusion, and hashes;
9. save a compact manifest and summary.

8.3 Operational output scale

The validated operational run is `operational_v1_1cafd65`. Its outputs are shown in Table 10.

Table 10. Validated operational prediction outputs.

Quantity	Value
Repository prediction rows	82
Predicted future growth surges	12
Mean four-week forecast	945.6431
Recommendation users	151,142
Recommendation catalog size	145
Top- K	10
Expected recommendation rows	1,511,420

The repository-prediction count and recommendation-catalog size are intentionally different. The recommender can rank all 145 repositories in its fitted catalog, whereas repository-level forecasting and classification require a complete recent weekly feature history and all frozen input columns. At the operational cutoff, 82 repositories satisfied that stricter repository-prediction contract.

8.4 Database write policy

The tracked Phase 2 database is immutable during operational prediction. New outputs are written under:

```
database/generated/github_herd_phase3_predictions.db
```

The output database records run metadata, repository forecasts, growth probabilities, binary predictions, user recommendations, rank, model score, model identifier, and generation time. The generated file is intentionally excluded from Git because it is approximately 277 MB; compact summaries and samples remain tracked.

8.5 Read-only delivery validation

The command below validates the complete frozen inference contract without writing predictions:

```
./run_phase3.sh --batch-size 5000
```

A successful run reports `passed_no_files_written`, confirms that protected files are unchanged, and records that models were not retrained and final-test scripts were not rerun.

9 MLOps, Automation, and Deployment

9.1 Phase 2 pipeline and GitHub Actions

The original `pipeline.py` executes database import, SQL queries, loading, EDA, preprocessing, and feature engineering sequentially using the same Python interpreter. The Phase 2 GitHub Actions workflow verifies that this data pipeline can run in a clean environment.

9.2 Prefect orchestration

Prefect was used as the structured workflow-orchestration layer [6]. The Phase 3 Prefect flow is `github-herd-phase3-safe-orchestration`. Its tasks verify frozen inputs, validate MLflow evidence, validate operational inference, and optionally execute a new write-enabled operational run. The default is deliberately read-only.

A Prefect deployment named `phase3-read-only-validation` was registered with a local Process work pool. A real worker-triggered run reached the states Scheduled, Pending, Submitting, Running, and Completed. The flow explicitly records:

- `models_retrained = false;`
- `final_test_scripts_rerun = false;`
- write mode disabled unless explicitly authorized;
- no schedule, so the deployment must be triggered manually.

9.3 MLflow model management

A separate dependency environment was used for MLflow 3.14.0 [5] and Prefect 3.7.8 [6]. The MLflow registration script verifies hashes and sample inference before registering the frozen models. It does not train or reopen final tests.

Registered model names include:

- `github-herd-growth-surge-classifier;`
- `github-herd-four-week-forecaster;`
- the frozen repository recommender.

Each selected model version receives the alias `champion`. Parameters, historical validation metrics, final-test metrics, signatures, artifacts, and freeze evidence are logged. The operational run is logged separately with row counts and database validation metrics.

9.4 Docker packaging

Docker was used to package the frozen runtime closure into a portable image [7]. The Phase 3 Dockerfile uses `python:3.12-slim` and installs only the minimal inference dependencies. The image contains the selected models, required data closure, tracked evidence, validator, and operational inference code. It excludes virtual environments, Git internals, and the large generated database.

The container is hardened with:

- non-root user `appuser` with UID/GID 10001;
- read-only root filesystem at runtime;
- no network during validation;
- all Linux capabilities dropped;
- no-new-privileges security option;
- CPU, memory, and PID limits;
- a small temporary filesystem mounted at `/tmp`.

The hardened container completed the full portable validator with exit code zero.

9.5 Kubernetes validation

A Kubernetes Job executes the same read-only container using a declarative batch-deployment specification [8]. The manifest disables service-account-token mounting, runs as non-root, uses a read-only root filesystem, drops all capabilities, and defines resource requests and limits. A local k3s cluster inside Colima executed the Job successfully. The resulting Pod completed with exit code zero, zero restarts, one successful Job completion, and no warning events.

9.6 Phase 3 GitHub Actions

The GitHub Actions delivery workflow [9] contains three independent jobs:

1. portable read-only validation in Python 3.12;
2. Docker image build and hardened container run;
3. Kubernetes YAML parsing and structural validation.

Two GitHub Actions runs completed successfully: one for the delivery-infrastructure commit and another after recording the CI evidence. This second run prevents the evidence commit itself from silently breaking the validated repository state.

Table 11. Validated automation and deployment components.

Component	Validation performed	Status
Phase 2 pipeline	End-to-end data loading, SQL, EDA, preprocessing, feature engineering	Passed
Prefect flow	Direct read-only flow, deployment registration, worker-triggered run	Completed
MLflow	Hash verification, model registration, version aliases, metric/evidence logging	Completed
Docker	Build, image inspection, hardened read-only execution	Passed
Kubernetes	Real Job and Pod execution on local k3s	1/1 succeeded
GitHub Actions	Portable validation, Docker job, Kubernetes manifest job	All jobs succeeded

9.7 Benefits and shortcomings of the automation choices

9.7.1 Benefits

- failures are detected from clean environments rather than only on the developer's machine;
- frozen artifacts and hashes make results auditable;
- Prefect provides task dependencies and structured run states;
- MLflow centralizes model versions and evidence;
- Docker reduces environment differences;
- Kubernetes demonstrates secure batch deployment;
- GitHub Actions provides automatic validation on pushes and pull requests.

9.7.2 Shortcomings

- the MLOps stack adds dependency and operational complexity;
- local Prefect and MLflow are not highly available production services;
- Docker images remain larger than a hand-built native binary;
- the Kubernetes artifact is a batch Job rather than a continuously served API;
- running local virtualization can consume substantial laptop resources;
- evidence and freeze rules require careful path and version management.

10 Integrated Technical and Business Analysis

10.1 What the three tasks provide together

The tasks answer different operational questions:

- forecasting estimates the magnitude of near-term attention;
- classification creates a compact early-warning list;
- recommendation connects individual users with repositories while controlling exposure concentration.

A technology-monitoring product could combine them in one dashboard. The forecast estimates expected growth, the classifier highlights uncertainty around surge regimes, and the recommender identifies users for whom the repository may be relevant.

10.2 Stakeholder analysis

Table 12. Potential stakeholders and decisions supported by the system.

Stakeholder	Decision supported	Important caution
Developers	Discover relevant tools and alternatives	Recommendation is based on stars, not direct task suitability
Maintainers	Prepare for support and visibility growth	Viral attention may not be predictable
Engineering managers	Monitor emerging technologies	Popularity is not equivalent to security or maintainability
Platform operators	Improve engagement and catalog exploration	Feedback loops can amplify popularity bias
Analysts and investors	Build technology trend watchlists	False positives require human verification
Researchers	Study collective attention and recommendation concentration	Sample is not the entire GitHub ecosystem

10.3 Business value

Potential value includes:

- reducing search cost for developers;
- increasing exposure of relevant long-tail projects;

- identifying repositories that may soon require additional maintainer capacity;
- supporting technology-scouting teams with quantitative watchlists;
- measuring whether a platform’s recommendation system is overly popularity driven;
- providing trend intelligence for open-source ecosystem strategy.

10.4 Cost of errors

A false positive growth alert costs analyst time and may cause unnecessary attention. A false negative may delay discovery of an important emerging technology. For recommendation, a false positive occupies one of ten scarce screen positions, while a false negative hides a repository the user would have valued. The correct threshold or ranking policy therefore depends on product goals.

10.5 Popularity feedback loops

Recommendation systems do not merely observe popularity; they can create it. Showing a repository increases the probability of additional stars, which may raise future model scores. The project’s popularity penalty, catalog coverage, novelty, and long-tail metrics provide a first defense against this feedback loop. A production platform should additionally use exposure logging, randomized exploration, counterfactual evaluation, and periodic bias audits.

10.6 Responsible interpretation

A star-growth forecast should not be described as a quality forecast. Repository adoption decisions also depend on licenses, vulnerabilities, maintainer responsiveness, bus factor, compatibility, documentation, and total cost of ownership. The appropriate interpretation is: “this repository is receiving or is expected to receive attention under patterns represented in the dataset.”

11 Limitations, Risks, and Future Work

11.1 Data limitations

Dataset scope

The dataset is sampled through selected topics rather than being a census of GitHub. Private activity is unobserved, stargazer collection may be capped, and repository metadata may be incomplete. Generalization beyond the sampled technology ecosystem is therefore uncertain.

Stars can be manipulated and may represent temporary attention. External events such as social-media posts, conference announcements, security incidents, or corporate releases are absent. These omitted variables are particularly relevant to the largest forecast errors.

11.2 Modeling limitations

The forecasting and classification tests contain 22 repositories. Although there are multiple time cutoffs, the effective number of independent entities is small. Confidence intervals should therefore account for repository clustering rather than treating all rows as independent.

The classifier's discrimination is moderate and its positive predictions are not sufficiently precise for unsupervised decisions. The forecasting model underpredicts extreme growth. The recommendation evaluation is offline and uses one held-out positive; it does not measure satisfaction, downstream use, or whether the user would have discovered the repository anyway.

11.3 Deployment limitations

SQLite is appropriate for reproducibility and local batch processing but not for high-concurrency online serving. Prefect and MLflow were validated locally rather than on managed infrastructure. The Kubernetes Job validates batch execution but does not provide an authenticated API, autoscaling service, observability dashboard, or continuous drift monitoring.

11.4 Ethical and ecosystem risks

A platform that predicts and recommends popularity may reinforce inequality between well-known and emerging maintainers. Malicious actors may attempt to inflate stars to gain recommendations. Repository suggestions could also expose users to unsafe or abandoned software. A production deployment should combine recommendation scores with security, maintenance, licensing, and trust signals.

11.5 Future work

Future development should include:

- broader and continuously updated repository collection;
- issue, pull-request, release, contributor, and dependency-network signals;
- external social and news signals for viral-event forecasting;
- prediction intervals and probabilistic forecasting;
- hierarchical models that share information across repositories while preserving entity-level uncertainty;

- temporal graph neural networks and sequence-aware recommenders;
- stronger cold-start content encoders;
- online A/B testing and interleaving experiments;
- exposure-aware and counterfactual recommendation evaluation;
- automated data and concept-drift monitoring;
- a production API and scheduled retraining policy;
- fairness, safety, and manipulation audits.

12 Conclusion

This project transformed raw GitHub API records into a reproducible analytical and operational system. The final database contains repository metadata, event-level stars, user interactions, daily and weekly time series, and herd-modeling features. The modeling stage compared multiple algorithms under leakage-aware temporal protocols and produced three frozen systems: a calibrated high-growth classifier, a baseline-residual Extra Trees forecaster, and a collaborative-graph recommender with a popularity penalty.

The results are useful but appropriately modest. The classifier provides a prioritization signal rather than a definitive decision. The forecaster captures ordinary short-term dynamics but struggles with rare viral bursts. The recommender achieves meaningful held-out recovery while using most of the catalog and maintaining high long-tail exposure and diversity. Together, these models provide a stronger view than any single popularity score.

The engineering contribution is equally important. Predictions are written back to an isolated SQLite database, frozen model versions are registered in MLflow, orchestration is validated with Prefect, execution is packaged in a hardened Docker image, a real Kubernetes Job succeeds, and GitHub Actions reproduces the portable validation from clean checkouts. The final system therefore satisfies the Phase 3 modeling and deployment requirements while documenting the scientific and operational safeguards necessary for trustworthy results.

A Reproducibility Commands

A.1 Main Phase 2 pipeline

```
1 python pipeline.py
```

A.2 Phase 3 model-development workflow

Phase 3 model development is organized as task-specific automated scripts rather than one monolithic training command. The following sequence reconstructs the pre-test model-development stages. Final-test scripts are deliberately excluded because the submitted models and one-time final-test results are frozen.

```
1 python scripts/build_phase3_datasets.py
2
3 python scripts/train_corrected_classification_models.py
4 python scripts/audit_and_freeze_classification_calibration.py
5
6 python scripts/train_forecasting_models.py
7 python scripts/audit_forecasting_baseline_residual_models.py
8 python scripts/freeze_forecasting_pretest.py
9
10 python scripts/train_recommendation_models.py
11 python scripts/audit_recommendation_model_search.py
12 python scripts/freeze_recommendation_design.py
13 python scripts/freeze_recommendation_pretest.py
```

These commands should only be used when reproducing a fresh development run. They should not overwrite the submitted frozen artifacts after the final test sets have been opened.

A.3 Portable read-only Phase 3 validation

```
1 ./run_phase3.sh --batch-size 5000
```

A.4 Operational validation without writes

```
1 python scripts/generate_operational_predictions.py \
2     --run-id delivery_validation \
3     --batch-size 5000 \
```

```
4 --validate-only
```

A.5 Write-enabled operational prediction

```
1 python scripts/generate_operational_predictions.py \  
2 --database-path database/generated/github_herd_phase3_predictions.db \  
3 --output-root outputs/phase3/operational_predictions \  
4 --run-id operational_<unique_id> \  
5 --batch-size 5000
```

A.6 Docker

```
1 docker build \  
2 --file Dockerfile.phase3 \  
3 --tag github-herd-phase3:latest \  
4 . \  
5 \  
6 docker run \  
7 --rm \  
8 --user 10001:10001 \  
9 --network none \  
10 --read-only \  
11 --tmpfs /tmp:rw,noexec,nosuid,nodev,size=128m \  
12 --cap-drop ALL \  
13 --security-opt no-new-privileges:true \  
14 --pids-limit 256 \  
15 --cpus 2 \  
16 --memory 2g \  
17 github-herd-phase3:latest \  
18 --batch-size 5000
```

A.7 Kubernetes

```
1 kubectl apply -f k8s/phase3-validation-job.yaml  
2 kubectl wait \  
3 --for=condition=complete \  
4 job/github-herd-phase3-validation \  
5 --timeout=10m  
6 kubectl logs job/github-herd-phase3-validation
```


A.8 Prefect

```
1 source venv_bonus/bin/activate
2 PREFECT_SERVER_ANALYTICS_ENABLED=false \
3 DO_NOT_TRACK=1 \
4 python flows/phase3_prefect_flow.py --batch-size 5000
```

A.9 MLflow

```
1 source venv_bonus/bin/activate
2 python scripts/register_frozen_models_mlflow.py --validate-only
```

B Frozen Artifacts and Git References

Table 13. Frozen artifact SHA-256 hashes.

Artifact	SHA-256
Classification model	7f66e845d0689c2c29583ec28d93218a602b951a3246a56767646d2cc48f5f23
Forecasting model	e728866f0b523a3393d1fe00cbf58b7500a02181866b337a4c1b78f8d2806294
Recommendation model	f2b178d1c888a04a5dbf15253e0d99b1a920e9230c549c8213e4f529f4c5e878
Phase 2 database	559803caa16b7f43a55cf6947f4eb580e92bb5a0ed7cd68ad0b581b960f88ee4

Validated Git references:

- repository: [github-herd-data-pipeline](#);
- Phase 3 branch: [github-herd-data-pipeline/tree/phase3](#);
- delivery commit: [d8f89bf941bcffe5a65738beef6a91155fe8e5e5](#);
- delivery tag: [phase3-delivery-validation-v1](#);
- final synchronized commit: [8a864558bc8ea79543a88bd587948cda3a3df087](#);
- CI evidence tag: [phase3-ci-evidence-v1](#);
- branch: [phase3](#).

C Repository Structure

```
1  .
2  |-- .github/workflows/
3  |   |-- data_pipeline.yml
4  |   `-- phase3_delivery.yml
5  |-- config/phase3_config.json
6  |-- data/
7  |   |-- raw/
8  |   |-- interim/
9  |   |-- processed/
10 |   `-- modeling/
11 |       |-- classification/
12 |       |-- forecasting/
13 |       `-- recommendation/
14 |-- database/
15 |   |-- github_herd.db
16 |   `-- generated/
17 |-- flows/phase3_prefect_flow.py
18 |-- k8s/phase3-validation-job.yaml
19 |-- models/
20 |   |-- classification/
21 |   |-- forecasting/
22 |   `-- recommendation/
23 |-- outputs/phase3/
24 |   |-- classification/
25 |   |-- forecasting/
26 |   |-- recommendation/
27 |   |-- operational_predictions/
28 |   |-- mlflow/
29 |   |-- prefect/
30 |   `-- delivery/
31 |-- scripts/
32 |-- Dockerfile.phase3
33 |-- pipeline.py
34 |-- prefect.yaml
35 |-- requirements.txt
36 |-- requirements-bonus-lock.txt
37 |-- requirements-phase3-runtime.txt
38 |-- run_phase3.sh
39 `-- README.md
```

References

- [1] University of Tehran, School of Electrical and Computer Engineering. *Introduction to Data Science Final Project, Phase 3: Tasks and Requirements*, 2026.
- [2] GitHub. *GitHub REST API Documentation*. <https://docs.github.com/en/rest>
- [3] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [4] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of KDD*, 2016.
- [5] MLflow Project. *MLflow Documentation*. <https://mlflow.org/docs/latest/>
- [6] Prefect Technologies. *Prefect Documentation*. <https://docs.prefect.io/>
- [7] Docker, Inc. *Docker Documentation*. <https://docs.docker.com/>
- [8] The Kubernetes Authors. *Kubernetes Documentation*. <https://kubernetes.io/docs/>
- [9] GitHub. *GitHub Actions Documentation*. <https://docs.github.com/en/actions>